

자동차IT융합학과 20241268 지윤주
자동차인공지능 이상헌교수님
2026/06/15

HW 5

Python 만을 이용한 CNN 구현과
MNIST 숫자 분류

1. 서론

1.1. 모델 구조

본 실험에서 사용한 CNN 알고리즘은 이미지의 공간적 특징을 추출하는 1개의 Convolution 계층과 최종 클래스를 분류하는 2개의 Affine 계층으로 구성되어 있다. 입력된 이미지 데이터는 Convolution과 Activation, Max pooling을 거쳐 벡터로 변환된 후 두 개의 Affine 계층과 Softmax를 통과하며 최종적인 숫자 분류 결과로 출력된다.

1.2. 실험 조건

1) 데이터셋

MNIST dataset

28×28 grayscale image

10개 클래스 (0~9)

Training set: 5,000

Validation set: 1,000

Test set: 1,000

2) 통제 변인

Input: (1, 28, 28)

Convolution layer 1

filter 수: 30

filter 크기: 5×5

stride: 1

padding: 0

Activation: ReLU

Pooling: 2×2 max pooling

Affine(hidden) layer: 100

Output layer: 10

Output activation: Softmax

Loss: Cross-Entropy Loss

Optimizer: SGD

Learning rate: 0.01

Batch size: 100

Epoch: 10 또는 20

3) 조작 변인

filter 수 : 10 / 30 / 50

filter 크기 : 3×3 / 5×5

Maxpooling : True / False

2. 본론 - 실험 결과 및 분석

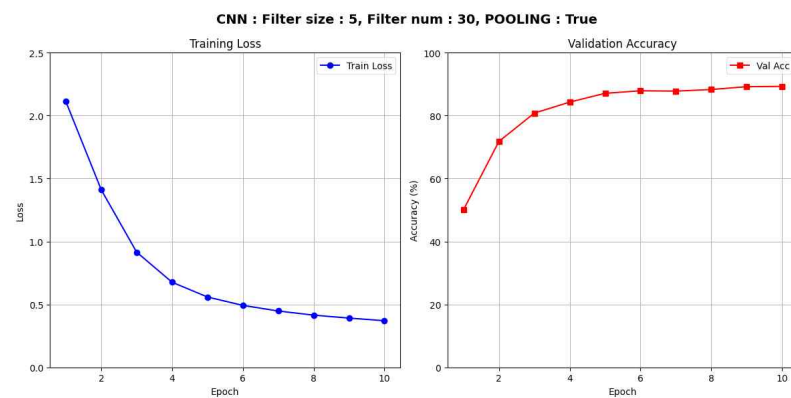
2.1. 실험 결과 표

* train acc, val acc는 마지막 10th epoch의 값을 기록

* 연두색 칸은 기본 BASELINE 조건임을 표시

Experiment	Setting	Train Acc	Val Acc	Test Acc	Comment
Filter 수	10	89.94	89.80	89.10	1분, 초기 val값 낮음
Filter 수	30	90.28	89.30	89.80	4분
Filter 수	50	91.18	90.80	90.80	7분, test acc 최고
Filter 크기	3*3	87.56	87.60	86.80	4분
Filter 크기	5*5	90.28	89.30	89.80	4분
MAXPOOLING	True	90.28	89.30	89.80	4분
MAXPOOLING	False	92.78	90.40	89.30	2분, 초기 val값 높음

2.2. Baseline Acc 추적 (Filter num = 30, filter size = 5*5 , maxpooling = True)



```
CNN : Filter size : 5, Filter num : 30, POOLING : True
Epoch [1/10] Loss: 2.1113, Train Acc: 30.78%, Val Acc: 50.10%
Epoch [2/10] Loss: 1.4100, Train Acc: 64.18%, Val Acc: 71.80%
Epoch [3/10] Loss: 0.9152, Train Acc: 78.40%, Val Acc: 80.80%
Epoch [4/10] Loss: 0.6763, Train Acc: 83.28%, Val Acc: 84.30%
Epoch [5/10] Loss: 0.5591, Train Acc: 85.44%, Val Acc: 87.10%
Epoch [6/10] Loss: 0.4925, Train Acc: 87.12%, Val Acc: 87.90%
Epoch [7/10] Loss: 0.4478, Train Acc: 88.32%, Val Acc: 87.80%
Epoch [8/10] Loss: 0.4149, Train Acc: 89.06%, Val Acc: 88.30%
Epoch [9/10] Loss: 0.3914, Train Acc: 89.80%, Val Acc: 89.20%
Epoch [10/10] Loss: 0.3706, Train Acc: 90.28%, Val Acc: 89.30%
```

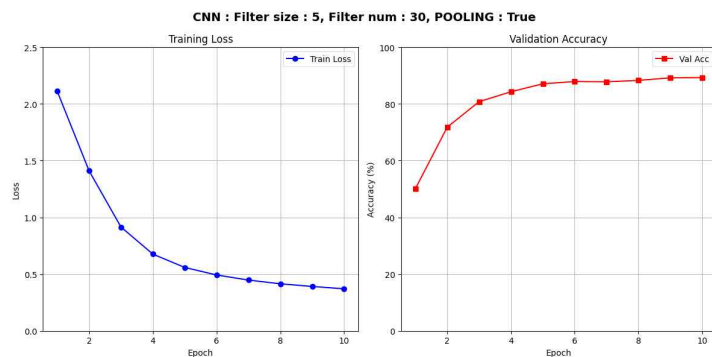
```
CNN : Filter size : 5, Filter num : 30, POOLING : True
Test Accuracy: 89.80%
```

2.2. Filter num 비교 (Filter size = 5*5 , MAXPOOLING = True)

1) filter num = 10



2) filter num = 30 (BASELINE)



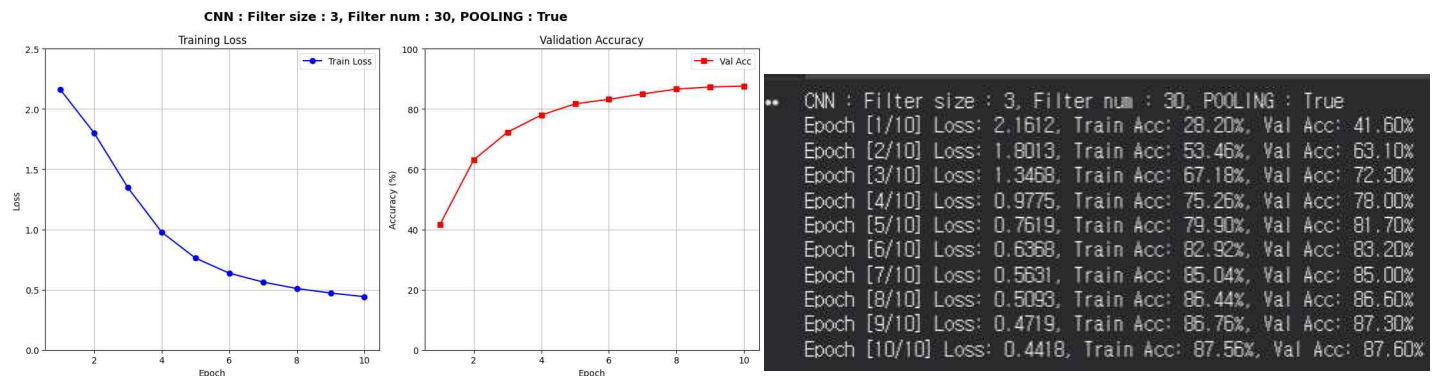
3) filter num = 50



- filter 수가 너무 적거나 많을 때 어떤 차이가 나타났는가?
필터 수가 적을 때는 파라미터 수가 적어서 연산이 빠르지만 가장 낮은 성능을 보였다. 반면 필터 수가 많을 때에는 연산 시간은 크게 늘어났지만 풍부한 특징을 학습해서 가장 높은 정확도를 달성했다.
- feature extraction 능력에 어떤 영향을 준다고 생각하는가?
필터 수는 입력 이미지에서 추출할 수 있는 특징의 수를 결정하므로 필터가 많을수록 다양한 특징을 추출해서 표현력을 향상시킬 수 있다.

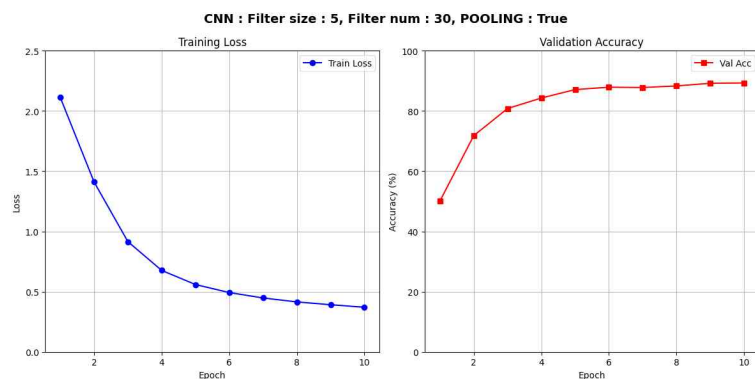
2.3. Filter size 비교 (Filter num = 30 , MAXPOOLING = True)

1) Filter size = 3*3



```
... CNN : Filter size : 3, Filter num : 30, POOLING : True
Test Accuracy: 86.80%
```

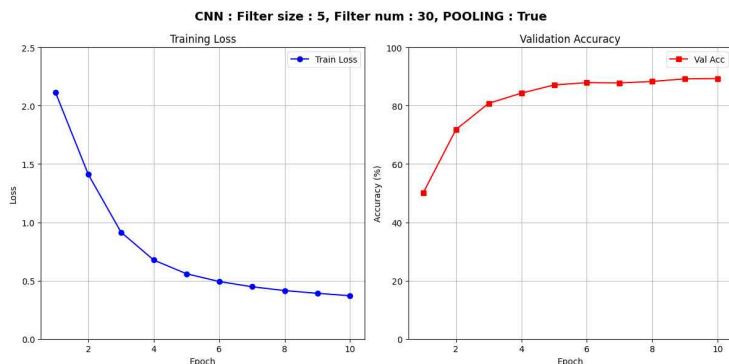
2) Filter size = 5*5 (BASELINE)



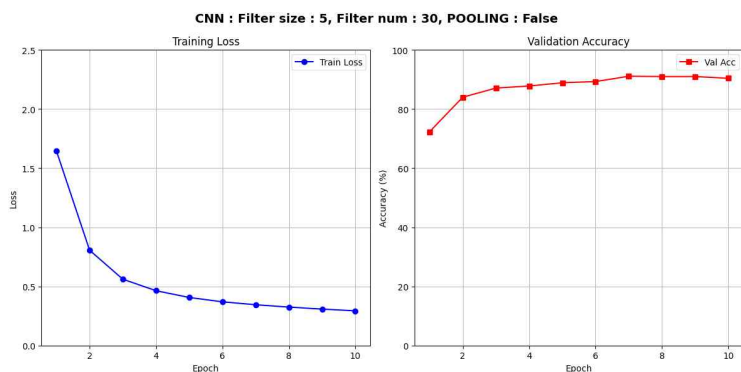
- 작은 filter와 큰 filter는 어떤 차이를 보였는가?
작은 필터는 국소적인 영역을 촘촘하게 훑지만 출력 해상도가 커서 뒤쪽 은닉층의 연산 부담이 컸다. 큰 필터는 정보를 비교적 빠르게 압축하는 특징이 있었다.
- MNIST와 같은 단순 이미지에서는 어떤 크기가 더 적절하였는가?
해상도를 적절히 축소하고 단순 숫자 형태를 잘 포착한 큰 필터 (5*5)가 더 적절했다.

2.4. Maxpooling 여부 비교 (Filter num = 30 , Filter size = 5*5)

1) Maxpooling = True (BASELINE)



2) Maxpooling = False



```
CNN : Filter size : 5, Filter num : 30, POOLING : False
Epoch [1/10] Loss: 1.6469, Train Acc: 49.04%, Val Acc: 72.30%
Epoch [2/10] Loss: 0.8059, Train Acc: 78.80%, Val Acc: 84.00%
Epoch [3/10] Loss: 0.5610, Train Acc: 84.96%, Val Acc: 87.10%
Epoch [4/10] Loss: 0.4636, Train Acc: 87.96%, Val Acc: 87.80%
Epoch [5/10] Loss: 0.4064, Train Acc: 89.30%, Val Acc: 88.90%
Epoch [6/10] Loss: 0.3694, Train Acc: 90.22%, Val Acc: 89.30%
Epoch [7/10] Loss: 0.3449, Train Acc: 90.98%, Val Acc: 91.10%
Epoch [8/10] Loss: 0.3250, Train Acc: 91.84%, Val Acc: 91.00%
Epoch [9/10] Loss: 0.3081, Train Acc: 92.12%, Val Acc: 91.00%
Epoch [10/10] Loss: 0.2934, Train Acc: 92.78%, Val Acc: 90.40%
```

```
... CNN : Filter size : 5, Filter num : 30, POOLING : False
Test Accuracy: 89.30%
```

■ pooling이 성능 또는 안정성에 어떤 영향을 주었는가?

pooling = false 상태에서는 train acc 대비 test acc가 뚝 떨어지는 overfitting 현상이 발생했다. 따라서 pooling은 노이즈를 억제하여 일반화 성능과 학습 안정성을 확보하는 역할을 했다.

■ pooling이 feature map 크기와 연산량에 어떤 영향을 주는가?

maxpooling은 가로 세로 해상도를 절반으로 줄여 전체 크기를 1/4로 압축한다. 따라서 다음 FC 층으로 입력되는 노드가 크게 감소해서 전체의 연산량을 줄여준다.

3. 결론

1. HW3의 fully connected network와 비교할 때 CNN의 성능은 어떠하였는가?

최종 ACC 값만 비교하면 HW3의 FC MLP가 CNN보다 더 높은 성능을 보였다.

이는 MNIST 데이터셋 자체가 공간적 왜곡이 적고 매우 단순하여 잘 최적화된 MLP로도 충분히 고성능을 낼 수 있는 반면, 이 실험의 CNN은 단일 conv구조의 한계와 별도의 최적화 튜닝이 포함되지 않아 성능이 제한적이었을 것이다.

2. filter 수가 성능에 어떤 영향을 주었는가?

필터 수가 많을수록 다양한 시각적 특징을 추출하여 모델의 표현력이 커지기 때문에 수렴 속도가 빨라지고 test acc 가 향상되는 영향을 주었다.

3. filter 크기 변화에 따라 어떤 차이가 나타났는가?

필터 크기가 커질수록 굵직한 형태적 특징을 빠르게 잡아내어 높은 성능을 냈고, 필터 크기가 작을수록 촘촘하게 보는 대신 출력 해상도가 커져 파라미터가 급증하고 연산 속도가 느려졌다.

4. pooling은 왜 필요한가?

Pooling은 사소한 변화나 노이즈를 무시하고 핵심 특징만 남겨 과적합을 방지하며, 해상도를 줄여 뒤쪽 은닉층의 파라미터수와 연산량을 절감해준다.

5. CNN이 MLP보다 이미지 분류에 적합한 이유는 무엇인가?

MLP는 이미지를 1차원으로 펼치면서 픽셀 간의 공간적 연관성을 잃어버리지만, CNN은 필터를 통해 이미지의 공간 구조와 픽셀 간의 관계를 유지하며 특징을 추출하기 때문에 더 적합하다.

7. 가장 성능이 좋았던 CNN 구조는 무엇이었는가?

본 실험에서 가장 성능이 좋았던 구조는 filter num=50, filter size=5*5, maxpooling=True 조건이었다. (test acc = 90.80%)